

Chapter 6

Deadlocks

Locks fix races and create the next problem: threads that wait for each other forever. With two teams adding locks to shared SWEB structures — the thread list, the address space, the scheduler — your group has to agree on how locks are taken. This chapter gives the theory (Coffman’s conditions, graphs, prevention, avoidance, detection) and the practice (lock ordering, validators), plus the relatives of deadlock: livelock, starvation and priority inversion.

6.1 Definition and the four conditions

A set of threads is *deadlocked* if every thread in the set waits for an event that only another thread in the set can cause. The smallest example: thread 1 holds lock A and waits for B, thread 2 holds B and waits for A. A deadlock is possible only if all four *Coffman conditions* hold:

1. **Mutual exclusion:** a resource is held by at most one thread.
2. **Hold and wait:** a thread holds resources while waiting for more.
3. **No preemption:** resources cannot be taken away; the holder releases them voluntarily.
4. **Circular wait:** a cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_1$, each waiting for a resource the next one holds.

Break any one and deadlock becomes impossible.

6.2 Resource-allocation and wait-for graphs

Draw threads as circles and resources as squares; an edge resource $\rightarrow$ thread means “held by”, thread $\rightarrow$ resource means “waits for”. With one instance per resource, a *cycle means deadlock*. Collapsing the resources gives the *wait-for graph*: $T_i \rightarrow T_j$ if T_i waits for something T_j holds.

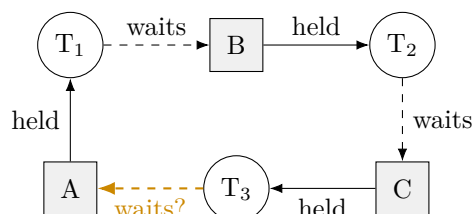


Figure 6.1: T_1 holds A and waits for B, T_2 holds B and waits for C, T_3 holds C. No cycle: T_3 can finish. If T_3 now requests A, the cycle $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ closes: deadlock.

6.3 Strategies

Prevention — make one condition impossible:

- mutual exclusion: share instead (read-only data, lock-free structures) — only for some resources;
- hold and wait: acquire everything at once, or release everything before waiting (trylock and back off — risks livelock);
- no preemption: take resources away (roll back a transaction) — rarely possible for locks;
- circular wait: **a global lock order**. If every thread takes locks in the same order, no cycle can form. This is the standard technique in kernels.

Avoidance — grant a request only if the system stays in a *safe state*: one from which some order lets every thread finish, even if all of them ask for their declared maximum. The *banker's algorithm* checks this. Example with two resource types A (10 units) and B (5 units):

	Allocation	Max	Need = Max - Alloc
P ₀	(2, 1)	(7, 3)	(5, 2)
P ₁	(3, 1)	(4, 2)	(1, 1)
P ₂	(2, 2)	(5, 4)	(3, 2)

Available = (10, 5) - (7, 4) = (3, 1). Safety check: P₁'s need (1, 1) ≤ (3, 1): it can finish and return its allocation, Work = (6, 2); then P₂ (3, 2) ≤ (6, 2), Work = (8, 4); then P₀ (5, 2). Safe, sequence P₁, P₂, P₀.

- P₀ requests (1, 0): pretend to grant, Available = (2, 1), P₀'s need (4, 2). P₁, then P₂, then P₀ can still finish: safe, grant.
- P₂ requests (1, 1) instead: Available would be (2, 0). No thread's need fits into (2, 0) — unsafe: P₂ must wait, although the units are free.

Avoidance needs to know maximum demands in advance; general-purpose kernels therefore rarely use it, but it is a classic exam topic.

Detection and recovery — let deadlocks happen, find the cycle in the wait-for graph, and break it (kill a thread, roll back). Databases do this. A kernel can at least *report* a deadlock instead of hanging silently.

Ignore it — the “ostrich algorithm”. For locks inside a kernel, the real answer is prevention by lock ordering, backed by tools that check the ordering.

6.4 Lock ordering in practice

- Document a hierarchy: “process lock before thread lock before address-space lock”. Code that holds a lower lock never acquires a higher one.
- Same-level locks (two accounts, two threads): order by a key — the account number, the address of the lock object.
- Never call out into unknown code (callbacks, other subsystems) while holding a lock; you do not know which locks it takes.
- Keep critical sections small, and never sleep while holding a lock somebody else needs to wake you.

Validators. Deadlocks depend on timing, tests rarely hit them. A *lock-order validator* records “A was held while B was acquired” edges during a normal run and reports a cycle as soon as some code path takes B then A — even if the deadlock never happened. Linux calls it *lockdep*; ThreadSanitizer has one built in; module 06, part C builds one.

6.5 Relatives of deadlock

Livelock threads keep running but make no progress — two threads that each trylock, fail, back off and retry in lockstep. Randomised back-off breaks the symmetry.

Starvation the system makes progress, but one thread never gets its turn: an unfair lock, a reader-preferring rwlock, a low priority under strict priority scheduling (chapter 9).

Priority inversion a high-priority thread waits for a lock held by a low-priority thread, while a medium-priority thread (which needs no lock) keeps the low one from running. The high thread effectively waits for the medium one. Mars Pathfinder (1997) reset itself because of this. *Priority inheritance*: the lock holder temporarily runs with the priority of its highest waiter.

In SWEB

Every `Lock` knows its holder (`held_by_`); every `Thread` keeps the locks it holds (`holding_lock_list_`) and the lock it waits for (`lock_waiting_on_`). Before a thread goes to sleep on a lock, `doChecksBeforeWaiting` can walk “waits for lock held by thread that waits for ...” and detect a real deadlock, and it catches recursive locking. `~Thread` asserts that a dying thread holds no locks. Interrupt handlers must not take ordinary locks at all: a handler waiting for a lock held by the thread it interrupted waits forever — a deadlock with a single thread.

Key points

- Deadlock needs all four Coffman conditions; break one.
- Single-instance resources: a cycle in the wait-for graph is a deadlock.
- Kernels prevent deadlock with a documented lock order.
- Banker’s algorithm: grant only if a safe sequence still exists.
- Validators find potential deadlocks from one run; SWEB’s lock lists support similar checks.
- Livelock, starvation and priority inversion are the neighbours to recognise.